

1.1.- Ejercicio 1: crear el proyecto Composer.

Orientaciones para el alumnado

En este ejercicio se trabajan los siguientes criterios de evaluación del RA 5: *desarrolla aplicaciones Web identificando y aplicando mecanismos para separar el código de presentación de la lógica de negocio.*

- a) Se han identificado las ventajas de separar la lógica de negocio de los aspectos de presentación de la aplicación.
- b) Se han analizado y utilizado mecanismos y frameworks que permiten realizar esta separación y sus características principales.

En esta primera etapa vamos a instalar y configurar un proyecto con *composer*. *composer* es un gestor de dependencias para PHP que permite automatizar la carga de dependencias y de clases. Este apartado viene explicado en los contenidos.

En este ejercicio tienes que realizar lo siguiente:

- En primer lugar, instala *composer* siguiendo lo indicado en los contenidos.
- Crea el proyecto `dwes04` con *composer*.
- **Namespace de tu aplicación:** decide como se denominará el espacio de nombres de tu aplicación (aplicable a tus propias clases).
- **Directorio de clases de tu aplicación:** Decide que directorio o carpeta en tu proyecto contendrá las clases de creación propia pertenecientes al espacio de nombres elegido. Las clases que irás creando tendrán que ir cargándose por autoloading PSR-4, por lo que debes configurar de forma adecuada el archivo `composer.json` para autoloading PSR-4.
- Añade como dependencia de producción la última versión de Smarty.
- Decide que directorio vas a utilizar para plantillas Smarty y créalo.

Recomendación

Una vez realizado lo anterior, es recomendable (pero no obligatorio) poner en práctica lo aprendido en esta unidad. Para ponerlo en práctica puedes crear una clase de prueba a tu elección, un script de punto de entrada inicial (por ejemplo, `http://localhost/dwes04/prueba2.php`) y dos plantillas Smarty, una con un formulario para recoger datos y otra para mostrar los resultados. Este proceso puedes hacerlo con clases de tu invención, por ejemplo:

- Crea una clase de prueba (en el directorio de clases y *namespace* elegido). Intenta que la clase tenga un constructor y al menos dos métodos de instancia (es decir, dos métodos no estáticos).
- Crea dos plantillas Smarty (una con un formulario y otra para mostrar la salida).
- Crea un script llamado `prueba1.php`, accesible en la ruta `http://localhost/dwes04/prueba1.php` que haga uso de la clase anterior (constructor y dos métodos de instancia) y de las plantillas Smarty. La clase creada deberá cargarse por autoloading PSR-4, así como Smarty.

1.2.- Ejercicio 2: creando el modelo.

Orientaciones para el alumnado

En este ejercicio se trabajan los siguientes criterios de evaluación del RA 5: *desarrolla aplicaciones Web identificando y aplicando mecanismos para separar el código de presentación de la lógica de negocio*.

- b) Se han analizado y utilizado mecanismos y frameworks que permiten realizar esta separación y sus características principales.
- g) Se han aplicado los principios y patrones de diseño de la programación orientada a objetos.

Siguiendo las líneas del ejemplo explicado en la unidad en el apartado 5, vamos a crear el modelo de una aplicación que sigue el patrón MVC sencillo (sin utilizar ningún *framework* complejo). Recuerda que el modelo es el que da soporte a los datos que maneja la aplicación.

Es necesario que las clases del modelo las separes en un subespacio (*namespace* espacio/subespacio;), tal y como explica los contenidos. Para ello debes ubicar las clases en un subdirectorio del directorio de clases elegido, al que a partir de ahora llamaremos "directorio del modelo". Tienes que prever que las clases siempre se cargarán usando autoloading PSR-4.

Nota importante 1: los métodos del modelo no deben enviar texto al navegador ni establecer conexión con la base de datos, esas no son sus atribuciones.

Nota importante 2: no se hará *include* ni *require* de estas clases o interfaces en ningún momento. Como se indica antes, las clases e interfaces deben cargarse por autoloading usando el cargador PSR-4 de *composer*, por lo que debes ubicarlas en un directorio del modelo e indicar el subespacio de nombres de forma correcta.

Paso 1) Crear la interfaz IGuardableXYZ (donde XYZ son tus iniciales).

La interfaz `IGuardableXYZ` tendrá 3 métodos:

- Método `_guardar`. El método `guardar` será un método que tendrá como parámetro una conexión `PDO` a la base de datos.
- Método `estático` `rescatar`. El método `rescatar` será un método que tendrá dos parámetros: una conexión `PDO` a la base de datos, y el identificador (`id`), que corresponde con la clave primaria en la tabla asociada.
- Método `estático` `borrar`. El método `borrar` será un método que también tendrá dos parámetros: una conexión `PDO` a la base de datos, y el identificador (`id`), que corresponde con la clave primaria en la tabla asociada.

El objetivo de estos métodos es unificar los métodos que se utilizan para gestionar los datos almacenados en la base de datos. En este caso solo usamos una tabla, pero cuando se usan muchas tablas tener métodos con los mismos nombres y parámetros facilita el trabajo.

Paso 2) Crear la clase Libro.

En el directorio del modelo deberás crear la clase `Libro`. Esta clase implementará obligatoriamente la interfaz `IGuardableXYZ`. Esta clase tendrá:

- Atributos privados para contener todos y cada uno de los datos de la tabla LIBROS especificada en el apartado "Descripción de la tarea", teniendo en cuenta que el valor por defecto de todos los atributos será `null`.

Dados a dichos atributos, esta clase tendrá los siguientes métodos `get` y `set` _____ :

- Método público `getId` que retornará el valor del atributo `id`. No debe existir el método `setId`.
- Método público `getFechaCreacion` que retornará el valor del atributo `fecha_creacion`. No debe existir el método `setFechaCreacion`.
- Método público `getFechaActualizacion` que retornará el valor del atributo `fecha_actualizacion`. No debe existir el método `setFechaActualizacion`.
- Método público tipo `get` para obtener el valor del resto de datos.
- Métodos públicos tipo `set` para establecer el resto de datos (`_setIsbn`, `setTitulo`, `setAutor`, etc.)
- Implementación del método `guardar`. Este método guarda (`INSERT`) el contenido en la base de datos asociado al libro _____, excepto `fecha_creacion` y `fecha_actualizacion` que se modifican automáticamente por la base de datos, e `id` que se genera automáticamente. ¡Cuidado! El campo `id` es autogenerado por la base de datos, esto significa que la primera vez que se guarda debe obtenerse el `id` autogenerado por `MySQL` con el método `lastInsertId` de la clase `PDO`, para después almacenar su valor en el

atributo id de la clase. Como resultado de la ejecución de este método se retornará:

- El número de filas creadas en la base de datos, si todo ha ido bien.
- Además, si todo ha ido bien, cambiará el valor del atributo `id` de la clase apropiadamente con el valor autogenerado por *MySQL*, y rellenará la fecha de creación y actualización conforme a lo que se ha generado en la base de datos ____.
- `false` o `-1` (a conveniencia) en caso de que la consulta haya fallado o se haya generado una excepción.
- Implementación del método estático `rescatar`. La función de este método es rescatar un registro con `id` _ indicado, y además, rellenar y retornar una instancia de la clase `Libro` con los datos rescatados. Retornará:
- retornará una instancia de la clase `Libro` con todos los datos rellenos con la información procedente de la base de datos _ .
- Nota importante: siendo `$libro` una instancia de la clase `Libro` creada dentro del método `rescatar`, al estar el método `rescatar` dentro de la clase `Libro` si es posible acceder al atributo `$libro->id` aunque este sea privado.
- `null` o `false` o `-1` o `-2` (a conveniencia) en caso de que la consulta haya fallado o se haya generado una excepción o no haya ningún `Libro` con el `id` indicado.
- Implementación del método estático `borrar`. La función de este método es borrar de la base de datos el registro con el `id` indicado de la tabla correspondiente _____, y como resultado de la ejecución se retornará:
- el número de filas eliminadas de la base de datos ____ .
- `false` o `-1` (a conveniencia) en caso de que la consulta haya fallado o se haya generado una excepción.

En los métodos que trabajan con la base de datos recuerda:

- Usar **consultas preparadas** cuando haya parámetros para las consultas.
- Usar `try-catch`.

Paso 3) Clase Libros.

La clase `Libros` contendrá métodos para manejar un conjunto de libros:

- Método estático `listarXYZ` donde `XYZ` serán tus iniciales. Este método tendrá como parámetro una conexión PDO válida a la base de datos y rescatará todos los libros retornando para ello obligatoriamente un **array con instancias de la clase `Libro`** _____, o bien `false` o `-1` (a conveniencia) en caso de que la consulta haya fallado o se haya generado una excepción.
- Este método tendrá un parámetro `boolean`, si el parámetro es `true` la lista se retornará en orden descendente en función de la `fecha_creacion`, en caso contrario, se retornará ordenada descendente en función de la `fecha_actualizacion`. _____

Para obtener instancias de la clase `Libro`, debes usar el método estático `rescatar` de dicha clase. En los métodos que trabajan con la base de datos recuerda:

- Usar **consultas preparadas** cuando haya parámetros para las consultas.
- Usar `try-catch`.

Recomendación

Puedes dejarlo para el final, pero en el Ejercicio 5 se pide que se pruebe el modelo creado. Es buena idea que le eches un vistazo al ejercicio 5 antes de pasar al ejercicio 3.

1.3.- Ejercicio 3: controladores y vistas (I)

Orientaciones para el alumnado

En este ejercicio se trabajan los siguientes criterios de evaluación del RA 5: *desarrolla aplicaciones Web identificando y aplicando mecanismos para separar el código de presentación de la lógica de negocio*.

- a) Se han identificado las ventajas de separar la lógica de negocio de los aspectos de presentación de la aplicación.
- b) Se han analizado y utilizado mecanismos y frameworks que permiten realizar esta separación y sus características principales.
- g) Se han aplicado los principios y patrones de diseño de la programación orientada a objetos.

IMPORTANTE: Crea una clase para contener los controladores. El nombre de la clase tiene que ser `ControladorXYZ` donde XYZ deben ser tus iniciales.

Nuevamente, al igual que ocurría en el modelo, es necesario que la clase del controlador la separe en un subespacio diferente (`namespace espacio/subespacio;`), tal y como explica los contenidos. Para ello debes ubicar las clases en un subdirectorio del directorio de clases elegido, al que a partir de ahora llamaremos "directorio del controlador". Debes prever que esta clase será cargada por autoloading PSR-4 a través de *composer*.

Normalmente, tal y como se muestra en el apartado 5 de la unidad 4, los controladores se implementan con métodos estáticos, por ejemplo:

```
public static function controladorDeEjemplo(Peticion $p, PDO $p, Smarty $s){ ...  
$s->display('template.tpl');}
```

Antes de continuar recuerda que:

- Los controladores reciben lo que necesitan por parámetro (conexión PDO, instancia de Smarty, etc.). Esos recursos (conexión PDO, instancia de Smarty, instancia de clase *Peticion*, etc.) se crean en el enrutador.
- Los controladores no muestran salida por si mismos, sino que lo hacen a través de la vista (en este caso plantillas Smarty), incluidos los formularios.
- Los controladores no realizan consultas a la base de datos, eso lo hacen en las clases del modelo.
- Los controladores SI verifican los datos recibidos vía POST o GET antes de pasarlos al modelo.
- Los controladores interconectan los componentes.

Nota: no es obligatorio el uso de la clase *Peticion* proporcionada. En caso de no usarla, deberás filtrar adecuadamente los datos recibidos vía POST y GET donde corresponda con *filter_input*.

Una vez hecho lo anterior tienes que implementar el controlador por defecto:

Controlador por defecto. Se mostrará un listado de libros ordenado donde se incluirán todos los datos de la base de datos para cada libro (incluido *id*, *fecha_creacion* y *fecha_actualizacion*). Este controlador tendrá un comportamiento diferente si se reciben datos vía POST o si no se reciben datos vía POST:

Caso A) NO se reciben datos vía POST o son datos incorrectos: mostrará un listado de todos los libros existentes ordenados por fecha de actualización,_____ y además, y un formulario para indicar si se desean ordenar por fecha de actualización o por fecha de creación. El formulario tendrá como destino la ruta por defecto (<http://localhost/dwes04/index.php>).

Caso B) SI se recibe vía POST el orden (ascendente por fecha de creación o ascendente por fecha de actualización). Cuando se recibe dicho día se mostrarán los libros en función del orden indicado. Si el orden recibido no es válido, se mostrará un mensaje de error y se listará como en el caso por defecto. En cualquier caso, se volverá a mostrar el formulario para ordenar los libros_____.

RECUERDA: los métodos del controlador no hacen consultas SQL ni muestran datos con "echo" o similar, deben hacerse a través de los métodos del modelo. Hacerlo significa que el ejercicio no es correcto.

Paralelamente a la implementación del modelo deberás crear el enrutador ([dwes04/index.php](http://localhost/dwes04/index.php)) y atender a la ruta por defecto: <http://localhost/dwes04/index.php>. En dicho enrutador deberás:

- Cargar el archivo *vendor/autoload.php* para que se produzca la carga automática de clases_____.
- Crear una instancia de PDO para pasarla al controlador por defecto si es necesario.
- Crear una instancia de Smarty para pasarla al controlador por defecto si es necesario.

- Ejecutar el controlador por defecto.

IMPORTANTE: todas las clases se cargarán por autoloading PSR-4, en ningún momento se realizará `include` o `require`. La única situación donde se podrá hacer uso de `include` o `require` será para cargar información de configuración de acceso a la base de datos desde el archivo `index.php`.

1.4.- Ejercicio 4: controladores y vistas (II)

Orientaciones para el alumnado

En este ejercicio se trabajan los siguientes criterios de evaluación del RA 5: *desarrolla aplicaciones Web identificando y aplicando mecanismos para separar el código de presentación de la lógica de negocio*.

- a) Se han identificado las ventajas de separar la lógica de negocio de los aspectos de presentación de la aplicación.
- b) Se han analizado y utilizado mecanismos y frameworks que permiten realizar esta separación y sus características principales.
- g) Se han aplicado los principios y patrones de diseño de la programación orientada a objetos.

Una vez que tienes implementado el controlador por defecto, vamos a implementar más funcionalidades (acciones), para lo que necesitarás: modificar el enrutador (`dwes04/index.php`), crear controladores adicionales para cada nueva acción y las plantillas Smarty necesarias. Las funcionalidades a implementar son las dos siguientes:

Añadir un nuevo libro al listado de libros

Eliminar un libro

IMPORTANTE: todas las clases se cargarán por autoloading PSR-4, en ningún momento se realizará `include` o `require`. La única situación donde se podrá hacer uso de `include` o `require` será para cargar información de configuración de acceso a la base de datos desde el archivo `index.php`.

1.5.- Ejercicio 5: documentación y prueba.

Orientaciones para el alumnado

En este ejercicio se trabajan los siguientes criterios de evaluación del RA 5: *desarrolla aplicaciones Web identificando y aplicando mecanismos para separar el código de presentación de la lógica de negocio.*

- h) Se ha probado y documentado el código.

En este ejercicio vamos a documentar y probar la aplicación.

Probar el modelo

Opción 1

Opción 2 (requiere investigación autónoma)

Documenta las clases del modelo y el controlador

Añade comentarios PHPDoc a las clases y métodos. El siguiente ejemplo es un comentario PHPDoc a nivel de clase:

```
/**
 * Clase que representa un ejemplo de documentación en PHP.
 *
 * Descripción detallada de la clase y su propósito.
 *
 * @author Tu Nombre
 */
class MiClase {}
```

Y el siguiente ejemplo es un comentario a nivel de método:

```
/**
 * Realiza .... --Descripción de la operación---.
 */
```



```
* @param int $parametro1 Descripción del primer parámetro.  
* @param string $parametro2 Descripción del segundo parámetro.  
* @return bool True si la operación fue exitosa, false de lo contrario.  
*/public function realizarOperacion($parametro1, $parametro2) {}
```